

Was ist git

- Versionsverwaltung
 - verschiedene Zustände von Code parallel
 - "sicheres" Arbeiten ohne Angst vor Verlusten
 - einfach ausprobieren auf der Basis der aktuellen Code-Version, wenn es nicht funktioniert, löschen
- Zusammenarbeit
 - mehrere Personen können an einem Projekt arbeiten
 - beliebig viele Branches
 - beliebig viele Arbeitskopien
- arbeiten mit github/gitlab
 - automatisierung
 - ci/cd
 - github actions
 - Projektmanagement
 - issues
 - milestones
 - epics
 - Verfügbarkeit von Code
 - kann überall hin geclost werden, wo git installiert ist

Installation

Windows

- <https://gitforwindows.org/>
- <https://git-scm.com/downloads/win>

Linux

- müsste schon drauf sein, falls nicht:
- <https://git-scm.com/downloads/linux>

Mac

- <https://git-scm.com/downloads/mac>

Wie funktioniert git

- git ist ein lokales Programm
- alles ist im lokalen .git Ordner gespeichert
- ueber urls koennen Verknuepfungen zu externen Directories oder Repos hergestellt werden
- alle Aenderungen sind zunaechst lokal und muessen manuell verteilt werden (push)

Erste Schritte

Konfiguration

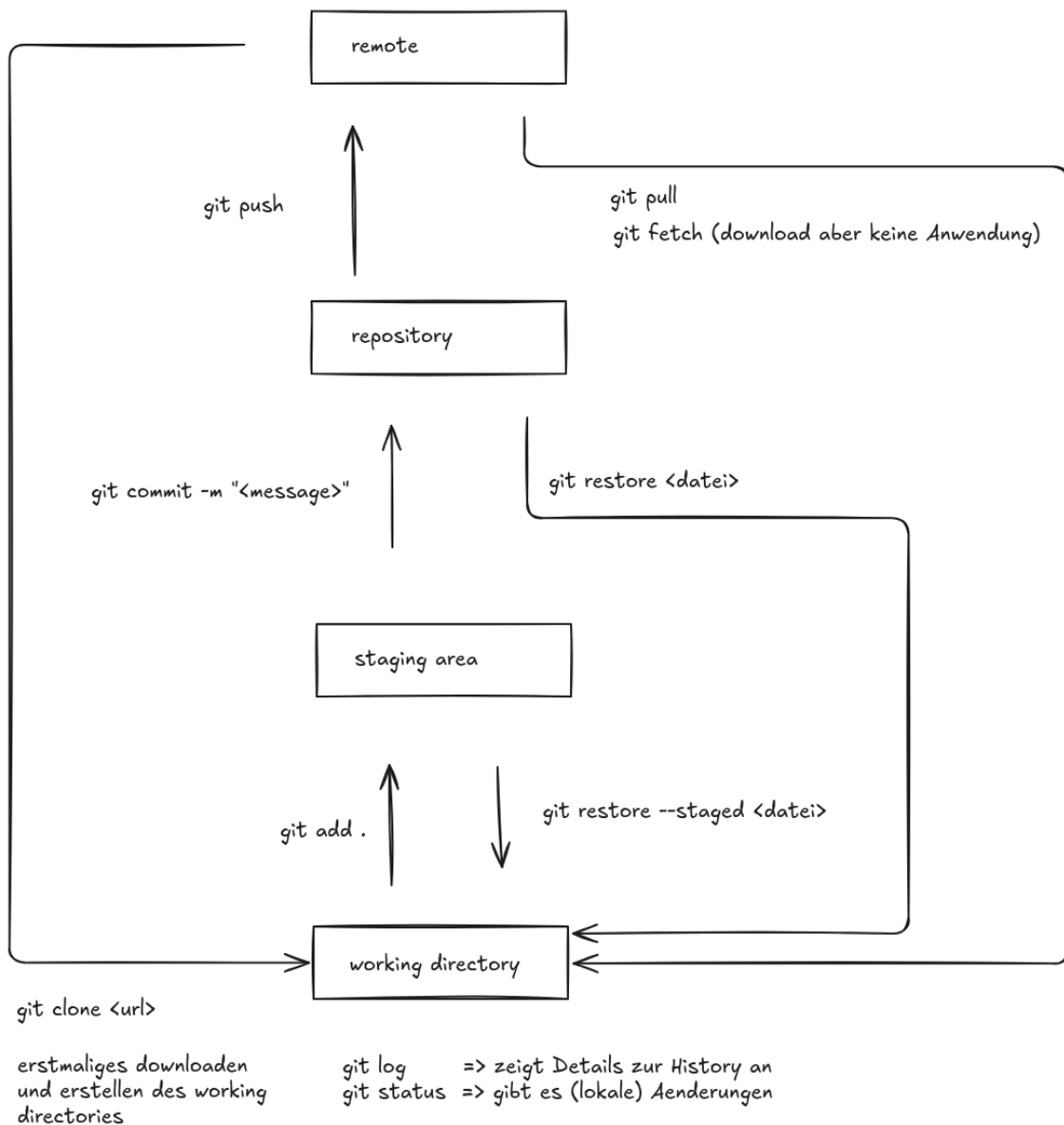
Wenn ihr git neu installiert habt, dann muesst ihr zunaechst ein paar globale Einstellungen vornehmen, sonst kann git nicht arbeiten und es funktioniert nicht. Ihr muesst git sagen, wie ihr heisst und welche email ihr hinterlegen wollt:

```
git config --global user.name "User Name"
```

```
git config --global user.email "user@email.com"
```

Das wird in den globalen, PC weiten, Konfigurationen gespeichert. Ihr koennt das spaeter auch noch aendern.

Workflows/Wichtige Commands



Working directory

Das eigentliche Arbeitsverzeichnis, in dem sich euer Code befindet. Dies liegt außerhalb des `.git` Ordners. Es gibt quasi zwei Möglichkeiten, ein Working directory zu erstellen. Entweder ihr habt schon einen Ordner/Verzeichnis mit eurem Code. Dann navigiert ihr mit eurem Terminal in dieses Verzeichnis und führt folgenden Befehl aus:

```
git init
```

Dies erstellt ein (noch) leeres `.git` Verzeichnis.

Mit

```
git clone <url>
```

erstellt kopiert ihr ein git repo von einem Remote aus, zum Beispiel, wenn ihr das Tutorium-Repo cloned. Ziel ist hier immer der Name des Repos (ihr koennt aber auch eigene Ziel-Verzeichnis-Namen vergeben, einfach dahinter schreiben).

Staging area

Die Staging area ist quasi eine Zwischenablage, in der ihr Dateien sammelt, die ihr in einem `commit` zusammenfassen wollt. Ein `commit` ist quasi eine Aenderungs-"Buchung", also ein Prozess, der einen History-Eintrag erzeugt, den ihr auch benennen koennt.

```
git add .
```

 fuegt alle geaenderten Dateien hinzu

```
git add <datei>
```

 fuegt die bestimmte Datei hinzu

Repository

Das ist die eigentliche History von git bzw. das `.git`-Verzeichnis. Hier werden alle Aenderungen und die History dazu gespeichert. Wie schon angesprochen, sind die einzelnen Aenderungsschritte sogenannte `commits`. Um ein commit zu erstellen:

```
git commit -m <message>
```

Mit `-m` benennt ihr die Aenderung bzw. beschreibt sie, sodass ihr spaeter am besten auch noch wisst, was ihr gemacht habt. Um ein `commit` erstellen zu koennen, braucht ihr Aenderung in der staging area, sonst funktioniert es nicht.

Remotes

Remotes sind externe Orte, an denen ihr `.git` Repos speichern und verwalten koennt. Bekannte Plattformen sind zum Beispiel Github, Gitlab, Bitbucket usw. Durch

```
git push
```

ladet ihr eure Aenderungen hoch. Mit

```
git pull
```

holt ihr euch Aenderungen vom Remote und wendet sie direkt an. Falls ihr nur schauen wollt, ob es Aenderungen gibt und welche, dann koennt ihr auch erstmal

```
git fetch
```

ausfuehren. Dies laedt erstmal alle Aenderungen herunter, wendet sie aber nicht an, und ihr koennt in aller Ruhe schauen, was sich alles geaendert hat, ohne das etwas "kaputt" gehen kann.

Branches

Git bietet die Moeglichkeit, beliebig viele Kopien von einem bestimmten Startpunkt (commit) aus, zu erstellen, sogenannte branches. Standardmaessig hat git nur einen branch, genannt `main` (aelter `master`). Mit

```
git branch <name>
```

erstellt ihr eine Kopie des gesamten Projekts vom aktuellen Stand (letztes commit) und speichert das quasi unter einem namen ab. Nun koennt ihr dort weiterarbeiten, ohne das main (die Hauptversion) veraendert wird. Der Workflow ist im Prinzip gleich, ihr muesst nur schauen, dass ihr auf dem richtigen "Branch" seid. Ueber

```
git switch <branch-name>
```

koennt ihr den branch wechseln und ueber

```
git branch
```

den aktuellen branch anzeigen lassen. Wenn ihr dann die Aenderungen wieder zusammenfuehren wollt, koennt ihr einen sogenannten `merge` ausfuehren. Dieser merge fuehrt zwei branches zusammen:

```
git merge <branchname> -m "<message>"
```

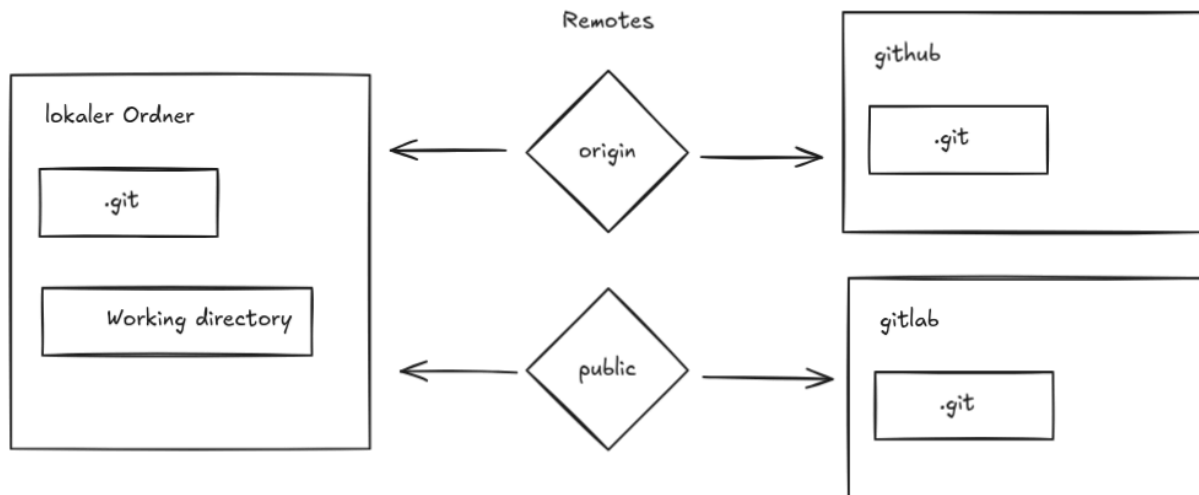
Hier gibt es einmal ein Ziel und eine Quelle. Das Ziel ist immer der Branch, auf dem ihr gerade seid, zum Beispiel `main`. Der angegebene Branch-Name ist der branch, der zu dem branch, auf dem ihr gerade seid, hinzugefuegt werden soll.

Sogenannte `merge Konflikte` koennen entstehen, wenn an der gleichen Stelle zwei Aenderungen stehen, dann muesst ihr entscheiden, welche Variante die aktuelle ist.

Aufbau lokal <--> remote

Git funktioniert, wie schon beschrieben, primär lokal. Falls ihr eine Verbindung zu einem sogenannten Remote haben wollt, also einem `.git` Verzeichnis, dass

woanders liegt, zum Beispiel auf Github oder Gitlab, dann sieht das so aus:



Origin und Public sind dann einfach nur namen fuer die eigentliche Verbindungs-Url zu den jeweiligen [Remote-Connections](#). Ihr koennt beliebig viele Remote connections hinzufuegen, ihr muesst dann nur beim pushen (siehe weiter unten) oder pullen darauf achten, wo ihr gerade mit arbeitet.